

Laboratorio 01 ARSW: Paralelismo, Arquitectura y Calidad desde el Día 1

María Paula Rodríguez Muñoz

Juan Andrés Suárez

Juan Pablo Nieto

Tomás Felipe Ramírez

1. Objetivo

El objetivo de este laboratorio es desarrollar e implementar un servicio paralelo para el cálculo eficiente de dígitos de Pi mediante técnicas de programación concurrente en Java. Los objetivos específicos incluyen:

- Implementar un algoritmo para calcular dígitos de Pi de manera secuencial.
- Paralelizar el algoritmo utilizando múltiples hilos (threads).
- Evaluar el desempeño comparativo entre versiones secuencial y paralela.
- Analizar la escalabilidad del algoritmo paralelo.
- Aplicar la Ley de Amdahl para predecir mejoras de desempeño.
- Crear una API REST que exponga la funcionalidad de cálculo de Pi.

2. Tabla de Tiempos

2.1. Metodología de Medición

Los experimentos de rendimiento se realizaron ejecutando el cálculo de 20 000 dígitos de Pi bajo diferentes estrategias de paralelización, manteniendo constantes las condiciones de ejecución.

Se evaluaron las siguientes configuraciones:

- **Ejecución Secuencial:** Uso de la estrategia `sequential`, con un único hilo de ejecución.
- **Ejecución Paralela:** Uso de la estrategia `threads`, variando el número de hilos.

- **Número de Hilos Evaluados:**

- 1 hilo
- `availableProcessors() = 8` hilos
- $2 \times \text{availableProcessors}() = 16$ hilos
- 200 hilos
- 500 hilos

- **Repeticiones:** Cada configuración fue ejecutada dos veces, y los resultados fueron utilizados para calcular tiempos promedio y métricas de speedup.

Los tiempos de ejecución se midieron en milisegundos (ms) y los resultados obtenidos se presentan en la Tabla correspondiente, la cual sirve de base para el análisis de rendimiento y la aplicación de la Ley de Amdahl.

2.2. Resultados Experimentales

Cuadro 1: Tiempos de ejecución (ms) – 20,000 dígitos

Configuración	Ejecución 1	Ejecución 2	Promedio	Speedup
Secuencial	14502	17336	15919	1.00x
Threads (8)	3823	23410	13616.5	1.17x
Threads (16)	2904	21411	12157.5	1.31x
Threads (200)	2542	7187	4864.5	3.27x
Threads (500)	2580	4198	3389	4.70x

3. Análisis de Resultados

3.1. Observaciones Principales

A partir de los resultados obtenidos para el cálculo de 20 000 dígitos de Pi, y considerando dos ejecuciones por cada configuración, se analizan los tiempos promedio con el fin de evaluar el impacto real de la paralelización.

1. **Comportamiento general del paralelismo.** Al utilizar múltiples hilos, el tiempo de ejecución se reduce en comparación con la versión secuencial. El tiempo promedio secuencial fue de 15 919 ms, mientras que las configuraciones paralelas presentan menores tiempos promedio, lo que confirma que el problema puede beneficiarse al usar el paralelismo. Sin embargo, la mejora no se mantiene para todas las configuraciones.

2. **Variabilidad en ejecuciones concurrentes.** Se presentan diferencias entre ejecuciones debido a la naturaleza concurrente del sistema y a factores externos, por lo que se utilizan valores promedio para el análisis.
3. **Relación entre número de hilos y núcleos disponibles.** Aunque el sistema dispone de ocho núcleos físicos, los mejores tiempos promedio no se obtienen necesariamente al usar el mismo número de hilos. Configuraciones con una mayor cantidad de threads presentan un mejor desempeño promedio, lo que sugiere que tener más hilos que núcleos en una cantidad moderada permite aprovechar mejor el procesador al ocultar latencias y distribuir la carga de trabajo de forma más eficiente.
4. **Límites de escalabilidad y overhead.** Aunque la configuración con 500 hilos presenta el mejor tiempo promedio, la mejora obtenida al aumentar la cantidad de threads no es proporcional. Esto se debe a que el sistema debe invertir tiempo adicional en administrar los hilos, lo que reduce las ganancias del paralelismo y establece un límite práctico en el rendimiento alcanzable.

3.2. Fórmula de Speedup

El speedup se define como la relación entre el tiempo de ejecución secuencial promedio y el tiempo de ejecución paralelo promedio para una configuración dada:

$$\text{Speedup}(p) = \frac{T_{\text{secuencial}}}{T_{\text{paralelo}(p)}} \quad (1)$$

En este experimento, el tiempo secuencial promedio fue de $T_{\text{secuencial}} = 15\,919$ ms. El mayor speedup promedio se obtuvo utilizando 500 hilos, con un tiempo promedio de $T_{\text{paralelo}} = 3\,389$ ms, lo que corresponde a un speedup aproximado de 4.70x.

4. Ley de Amdahl

4.1. Modelo Teórico

La Ley de Amdahl establece que el speedup máximo alcanzable por un programa paralelo está limitado por la fracción secuencial del código y se expresa como:

$$S(p) = \frac{1}{f + \frac{1-f}{p}} \quad (2)$$

donde f representa la fracción del programa que no puede paralelizarse y p es el número de procesadores o hilos utilizados.

4.2. Aplicación Experimental

A partir del speedup máximo promedio observado experimentalmente, cercano a 4.70x al utilizar 500 hilos, se puede estimar la fracción secuencial del algoritmo. Bajo el supuesto de que esta configuración se aproxima al límite práctico del sistema, se obtiene:

$$f \approx \frac{1}{4,70} \approx 0,213 \quad (21,3 \%)$$

Este resultado sugiere que aproximadamente el 21 % del tiempo total de ejecución corresponde a secciones no paralelizables, como la inicialización del proceso, la coordinación de los hilos y la recolección de resultados.

En consecuencia, el speedup máximo teórico del sistema estaría acotado por:

$$S_{max} = \frac{1}{f} \approx 4,7$$

lo cual es consistente con los resultados experimentales promedio obtenidos.

5. Conclusiones

1. La paralelización del algoritmo permitió reducir el tiempo de ejecución frente a la versión secuencial, aunque se observó una variabilidad significativa entre ejecuciones.
2. El análisis basado en valores promedio muestra que el speedup máximo alcanzado es cercano a 4.7x, en concordancia con las predicciones de la Ley de Amdahl.
3. Los costos asociados a la gestión de hilos limitan la escalabilidad al incrementar excesivamente el número de threads.
4. La arquitectura REST facilita el acceso concurrente al servicio de cálculo, proporcionando una solución desacoplada y flexible.